
Adaptive Saccadic Reservoir Imaging Networks: Co-Optimizing Visual Scanning and Critical Dynamics

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Reservoir computing is attractive for its fixed recurrent weights, efficient training,
2 and kernel-theoretic clarity, yet it struggles on images because static inputs lack
3 temporal structure. Hand-crafted raster scans or Poisson spike encodings ignore
4 2-D semantics, generate long redundant sequences that overwhelm fading memory,
5 fix the reservoir’s operating point, and yield brittle performance. We introduce
6 Adaptive Saccadic Reservoir Imaging Networks (ASRIN), which jointly learns an
7 image-to-sequence interface and self-tunes reservoir dynamics. ASRIN comprises:
8 (i) a differentiable saccadic encoder that selects a short sequence of foveal patches
9 via a learned policy; (ii) an edge-of-chaos self-tuning reservoir that modulates a
10 global gain online to keep the effective Lipschitz factor near unity; and (iii) a dual-
11 stream linear readout combining current patch and reservoir state. We further derive
12 a recurrent-kernel limit K_{ASRIN} that depends on the learned scan path, enabling
13 analysis and kernel ridge classification. A PyTorch implementation is provided. A
14 deliberately tiny MNIST pilot ($N = 128$, $T = 6$, two epochs) shows substantial
15 compute savings versus raster ESNs, learned saccades outperform a fixed pseudo-
16 raster, and mis-tuned gain control that motivates controller refinement. We detail
17 a full protocol across MNIST, Fashion-MNIST, and EMNIST with robustness,
18 dynamics, and kernel correspondence, evaluated over 10 seeds for statistical rigor
19 Dong et al. [2020], Ivanov and Michmizos [2021], Nestler et al. [2020].

20 1 Introduction

21 Reservoir computing fixes internal recurrent weights and trains only a linear readout, offering a
22 compelling blend of simplicity, efficiency, and theoretical grounding through recurrent kernels in
23 the large-width limit Dong et al. [2020]. This paradigm excels on temporal signals but faces a
24 fundamental mismatch with static images, which lack an intrinsic temporal axis. The prevailing
25 remedy is to impose a hand-crafted conversion—typically a raster scan or Poisson spike encoding.
26 These encodings discard 2-D structure, inflate sequence length, dilute fading memory, and lock the
27 reservoir’s operating point, often producing brittle behavior under natural perturbations.

28 Our objective is to bridge this gap: can an image–reservoir interface be learned so that (i) visual
29 evidence is revealed adaptively over time, and (ii) the reservoir’s dynamical regime self-organizes
30 near criticality, thereby improving accuracy and robustness while preserving the hallmark simplicity
31 of reservoir computing? Two challenges arise. First, the encoder must be differentiable and content-
32 adaptive under a fixed reservoir, calling for a policy over spatial glimpses that is trained jointly with
33 the readout. Second, performance is sensitive to dynamical parameters (spectral radius, leak, non-
34 linearity); maintaining near-critical dynamics online is hard yet crucial for information propagation
35 and memory Ivanov and Michmizos [2021], Orhan and Pitkow [2019]. Beyond engineering, there
36 is a theoretical gap: how do content-dependent input trajectories induced by attention reshape the
37 reservoir’s kernel limit Dong et al. [2020], Nestler et al. [2020]?

We propose Adaptive Saccadic Reservoir Imaging Networks (ASRIN), a framework that unifies learned visual attention with online criticality control and couples both to a recurrent-kernel analysis conditioned on the learned scan path. ASRIN is inspired by: recurrent kernels for random reservoirs Dong et al. [2020]; astrocyte-modulated self-organization to the edge of chaos Ivanov and Michmizos [2021]; and analyses showing that optimizing the input projection structure can markedly boost recurrent performance Nestler et al. [2020]. Rather than feeding all pixels, ASRIN learns a short sequence of informative foveal patches, preserving spatial semantics and reducing temporal burden. Simultaneously, it adjusts a global gain to stabilize the effective Lipschitz factor around unity, targeting a largest Lyapunov exponent close to zero. A dual-stream readout exposes the respective roles of instantaneous evidence and recurrent integration. Finally, we derive a closed-form recurrent-kernel construction that depends on the learned trajectory π *theta*, building a principled bridge between active perception and reservoir kernels Dong et al. [2020].

- **Differentiable saccadic encoder:** Learns task-driven patch sequences, drastically shortening inputs while preserving spatial structure.
- **Edge-of-chaos self-tuning:** An astrocyte-like global gain targets near-critical dynamics online, avoiding costly hyperparameter sweeps Ivanov and Michmizos [2021].
- **Dual-stream readout:** A linear classifier that combines the current patch and reservoir state, enabling early prediction and ablation of instantaneous versus memory contributions.
- **Trajectory-conditioned kernel:** A recurrent-kernel formulation, K_{ASRIN} , that depends explicitly on the learned scan path, enabling kernel ridge classification and finite-width comparisons Dong et al. [2020].
- **Open implementation and pilot:** A PyTorch pipeline and a feasibility pilot that validates end-to-end training and reveals controller tuning needs.

We validate feasibility with a tiny MNIST pilot ($N = 128$, $P = 8$, $T = 6$, two epochs, single seed) to verify execution under tight budgets. The pilot shows that learned saccades outperform a fixed pseudo-raster at comparable compute and that short sequences provide large compute savings over raster ESNs. The edge-of-chaos controller operated supercritically, reinforcing the need for a more conservative adaptation regime at small N and T . The full protocol targets MNIST, Fashion-MNIST, and EMNIST (balanced); evaluates robustness to noise, rotations, occlusions, and FGSM attacks; estimates dynamical metrics; and probes kernel-finite correspondence, all over 10 seeds with paired tests.

Relation to prior work spans recurrent kernels and structured reservoirs Dong et al. [2020], optimized input projections and unfolding analyses Nestler et al. [2020], astrocyte-inspired criticality Ivanov and Michmizos [2021], stability- and memory-aware recurrent design Orhan and Pitkow [2019], Rusch and Mishra [2021], on-sensor and active perception So et al. [2023], state-space models Hasani et al. [2022], robustness via dynamical stability Ding et al. [2024], and structured recurrent connectivity Tumma et al. [2023], Su et al. [2020]. Our approach differs by fixing recurrent weights, learning the encoder and readout only, maintaining online criticality, and providing a trajectory-conditioned kernel analysis.

Looking forward, we aim to execute the full evaluation suite, refine the gain controller for reliable near-criticality, extend to event-based sensors, and integrate structured or low-rank reservoirs within ASRIN Tumma et al. [2023], Su et al. [2020].

2 Related Work

Reservoir kernels and structured reservoirs. Random reservoirs converge to recurrent kernels in the infinite-width limit, enabling learning through kernel ridge regression and yielding convergence guarantees Dong et al. [2020]. Structured transforms preserve performance while improving computational efficiency Dong et al. [2020]. These lines assume exogenous, fixed input sequences. ASRIN departs by making the input sequence content-adaptive through a learned saccadic policy. This adaptivity changes the input statistics seen by the reservoir and, correspondingly, alters the induced kernel, which we capture through K_{ASRIN} .

Optimizing input projections and active sensing. Analyses that unfold recurrent dynamics into effective feed-forward temporal kernels emphasize that performance hinges on the interaction between

input projections, nonlinearity, and stimulus statistics Nestler et al. [2020]. On-sensor active recurrent processing reduces bandwidth and latency by acquiring information adaptively So et al. [2023]. ASRIN follows this spirit but keeps recurrent weights fixed: it learns where and when to look via a differentiable encoder, while the reservoir remains random and frozen.

Edge-of-chaos and dynamical self-tuning. Astrocyte-modulated plasticity can self-organize spiking liquids near the edge of chaos, improving performance and robustness without extensive hyperparameter sweeps Ivanov and Michmizos [2021]. Stability- and memory-aware designs reveal the benefits of balancing sensitivity with stability through non-normal dynamics and bounded gradient growth Orhan and Pitkow [2019], Rusch and Mishra [2021]. ASRIN adapts these ideas to a rate-based ESN by introducing a lightweight global gain controller aimed at near-unit effective Lipschitz constants.

Robustness and stability in neural dynamics. Stability considerations underpin robustness to noise and adversarial perturbations in spiking systems Ding et al. [2024]. The theory of continuous attractors highlights the role of near-neutral modes for persistence and sensitivity trade-offs Ábel Sági et al. [2024]. ASRIN’s gain controller operationalizes these principles by targeting near-zero Lyapunov exponents.

Broader recurrent architectures. Liquid and structural state-space models learn long dependencies with strong results across domains Hasani et al. [2022]. Low-rank and sparse recurrent connectivity improves robustness under closed-loop distribution shift Tumma et al. [2023]. Higher-order convolutional LSTMs capture complex spatiotemporal correlations Su et al. [2020]. Quantum recurrent networks explore alternative substrates Bausch [2020]. These methods typically train recurrent parameters end-to-end. In contrast, ASRIN retains fixed recurrent weights for simplicity and interpretability while learning the input path.

Random features and minimal learning. Random-feature MLPs and slot-machine parameterizations demonstrate strong baselines with fixed features and minimal learning Aladago and Torresani [2021]. We include such baselines to contextualize ASRIN’s benefits from learned saccades and controlled dynamics. Overall, to our knowledge, no prior work jointly learns a content-adaptive scan path, maintains near-critical reservoir dynamics online, and provides a trajectory-conditioned recurrent-kernel analysis in a single reservoir framework Dong et al. [2020], Nestler et al. [2020], Ivanov and Michmizos [2021].

3 Background

Problem setting. We address single-image classification for grayscale images of size H by W . To interface with a reservoir, each image I is transformed into a short sequence of foveal patches x_t of size P by P for $t = 1, \dots, T$. Each patch is vectorized to dimension $d = P^2$ and standardized per example. The goal is to choose T and the patch locations so that class-discriminative information is captured with minimal sequence length.

Reservoir computing. We employ a leaky echo-state reservoir with hidden state $h_t \in \mathbb{R}^N$. At time t , the preactivation is formed by a global-gain-scaled recurrent contribution plus a linear projection of the current input x_t . The new state is a convex combination of the previous state and the nonlinearity applied to the preactivation; we use $\tanh$ and leak α . Input and recurrent weights are fixed at random after initialization; only a linear readout is trained. This setting underlies connections to recurrent kernels in the large-width limit Dong et al. [2020].

Edge-of-chaos dynamics. Recurrent networks display a trade-off between sensitivity and stability governed by the largest Lyapunov exponent (LLE). Near zero, networks can preserve information without diverging, supporting both memory and robustness. Astrocyte-modulated spiking liquids self-organize toward this regime and improve performance Ivanov and Michmizos [2021]. We adopt an analogous mechanism for a rate-based reservoir by modulating a global gain to target an effective Lipschitz factor near one.

Active perception and learned trajectories. Performance of recurrent systems depends on the interaction between input statistics and nonlinearity Nestler et al. [2020]. Instead of a hand-crafted raster,

142 we learn a saccadic policy that reveals informative patches. This preserves spatial semantics and
143 shortens the sequence, mitigating fading-memory dilution and reducing compute.

144 **Kernel limit.** In the large-width regime and under standard Gaussian assumptions on random weights,
145 reservoir dynamics converge to a deterministic recurrent kernel Dong et al. [2020]. In ASRIN, the
146 kernel depends on reservoir hyperparameters and on the content-dependent sequence generated by
147 the learned policy. We use an arcsin approximation to the expected product of
148 $\tanh$ activations under a bivariate Gaussian to derive a closed-form recursion for state covariances,
149 culminating in a kernel that mirrors our dual-stream readout.

150 **Assumptions and diagnostics.** Reservoir weights and leak are frozen. The encoder is differentiable
151 via bilinear patch sampling and a Gumbel-Softmax policy over grid locations. We monitor dynamics
152 by estimating LLE through the action of local Jacobians on random tangent vectors and quantify
153 memory via linear reconstruction of delayed inputs from reservoir states. These diagnostics test
154 whether the controller achieves near-criticality and adequate short-term memory Orhan and Pitkow
155 [2019], Ivanov and Michmizos [2021].

156 **4 Method**

157 **4.1 Differentiable saccadic encoder**

158 The encoder constructs a short sequence of $P \times P$ patches. A policy
159 p_i
160 θ_i maps current visual evidence and provisional class logits to a distribution over a $G \times G$ grid
161 of candidate centers. Concretely, a small GRU receives the current patch embedding (obtained by a
162 linear layer and ReLU) concatenated with the previous-step class logits and outputs logits over G^2
163 grid cells. A Gumbel-Softmax with temperature
164 τ_i produces soft attention over the grid; the expected center is the weighted average of grid
165 coordinates. A $P \times P$ patch is sampled from the image at that center using bilinear interpolation
166 and standardized per example. The sequence length is kept short (around $T \approx 30$ in the main
167 configuration). Two regularizers improve learning: an entropy bonus early in training to avoid
168 premature attention collapse, and an $L2$ penalty on center displacement to suppress jerk.

Algorithm 1 Saccadic encoder with Gumbel-Softmax centers

Input: image I , grid coordinates $\mathcal{G} = \{g_k\}_{k=1}^{G^2}$, temperature τ , steps T
Initialize GRU state s_0 , logits ℓ
 $\ell = 0$
for $t = 1, \dots, T$ **do**
 Embed previous patch: $e_{t-1} \leftarrow f_{\text{textemb}}(x_{t-1})$ (**use zeros for** $t = 1$) $\text{GRUupdate} : s_t \leftarrow \text{GRU}([e_{t-1}; \ell_{t-1}], s_{t-1})$
 Grid logits: $z_t \leftarrow Ws_t + b$
 Gumbel-Softmax: $w_{t,k} \leftarrow \text{Softmax}(z_t + g_{t,k})$ where $g_{t,k} \sim \text{Gumbel}(0, 1)$
 Expected center: $c_t \leftarrow \sum_k w_{t,k} g_k$
 Sample patch: $x_t \leftarrow \text{BilinearSample}(I, c_t, P)$; standardize x_t
 Provisional logits: $\ell_t \leftarrow W \text{read}(x_t; h_{t-1})$ (**stop-grad through reservoir**)
end for
Output: sequence x
 $\frac{t}{T}$
 $t=1$

169 4.2 Edge-of-chaos self-tuning reservoir

170 We use a sparse echo-state network with
171 $\tanh$ units, leak
172 α , and spectral-radius-normalized random recurrent weights. Denote the global gain by g and
173 spectral radius by
174 ρ (initialized to 1). The dynamics are

$$a_t = g; Wh_{t-1} + Ux_t, \quad h_t = (1 - \alpha)h_{t-1} + \alpha \tanh(a_t).$$

175 We estimate the mean slope of the nonlinearity by m

176 $m =$
177 $\frac{1}{N} \sum_i (1 - \tanh^2(a_{t,i}))$ and track an exponential moving average
178 $\bar{m}$
179 $\bar{m} =$
180 $\bar{m} = \alpha \bar{m} + (1 - \alpha) m$
181 $\bar{m}$
182 $\bar{m}$
183 $\bar{m}$. A proxy for the effective Lipschitz factor is

$$L_t = \alpha \bar{m} + (1 - \alpha).$$

184 The astrocyte-like gain updates multiplicatively: g
185 $g \leftarrow g \cdot \bar{m}$

```

186  $\dot{c}$ 
187  $\exp($ 
188  $\eta$ 
189  $\dot{c}(1 - L$ 
190  $t))$ , with small step size
191  $\eta$  and clamping to a safe interval.

```

Algorithm 2 Global-gain adaptation toward the edge of chaos

Input: leak α , spectral radius ρ , step size η , smoothing β , clamp interval $[g_{\min}, g_{\max}]$

Initialize $g \leftarrow 1$, b_{arm}

for each time step t **do**

Compute preactivation a_t and update h_t

Instantaneous slope mean: $m_t \leftarrow \frac{1}{N} \sum_i (1 - \tanh^2(a_{t,i}))$

EMA: $b_{arm} \leftarrow \beta b_{arm} + (1 - \beta) m_t$

Lipschitz proxy: $L_t \leftarrow \alpha g + \rho b_{arm}$

Gain update: $g \leftarrow \dot{c} \exp(\eta \dot{c} (1 - L_t))$

Clamp: $g \leftarrow \min(\max(g, g_{\min}), g_{\max})$

end for

4.3 Dual-stream linear readout

The classifier is linear in the concatenation of the current patch vector and the reservoir state, $[x_t; h_t]$. We train a final-step readout on $[x_T; h_T]$ and optionally log intermediate predictions to study early exits. Removing either stream enables ablations that separate instantaneous evidence from recurrent integration.

199 4.4 Training protocol

200 Only

201 π

202 θ and the linear readout are learned with Adam; the reservoir remains fixed. Gradients flow through
 203 differentiable sampling and through the unrolled reservoir states but not into reservoir parameters.
 204 Learning rates are separated for policy and readout, and the Gumbel-Softmax temperature
 205 τ is annealed from approximately 1.0 to 0.3 over training. Light data augmentation uses small
 206 translations and rotations to improve invariance.

207 4.5 Recurrent-kernel limit

208 For examples i and j , define the per-time input Gram C^x

209 $C_t^x(i, j) =$

210 $\frac{1}{d} \langle x_t^i, x_t^j \rangle$

211 $\langle x_t^i, x_t^j \rangle$

212 $\langle x_t^i, x_t^j \rangle$

213 $\langle x_t^i, x_t^j \rangle$

214 $\langle x_t^i, x_t^j \rangle$, with $d = P^2$. Initialize the state covariance C^h

215 to zero. The preactivation covariance at time t is

$$S_t(i, j) = g^2 \rho^2; C_{t-1}^h(i, j) + \sigma_{\text{textin}}^2; C_t^x(i, j),$$

216 where

217 σ

218 σ_{textin} controls input scaling. Using an arcsin approximation for

219 $\tanh$, the covariance of nonlinear activations is

220 Φ

221 $C_t^x(i, j) =$

222 κ

223 $\text{textarcsin}(S$

224 $t; q$

225 i, q

226 j), where q

227 i and q

228 j are the corresponding variances and

229 κ

230 textarcsin is the normalized arcsin function applied elementwise. The state covariance updates as

$$C_t^h = (1 - \alpha)^2 C_{t-1}^h + \alpha^2 \Phi_t,$$

231 neglecting cross terms for simplicity. To mirror the dual-stream readout, the final kernel is

$$K_{\text{mathrmASRIN}} = C_T^h + \lambda \sum_{t=1}^T C_t^x,$$

232 with

233 λ tuned on validation. Kernel ridge regression on K

234 mathrmASRIN enables comparison to finite reservoirs driven by the same learned sequences Dong
 235 et al. [2020].

Algorithm 3 Recurrent-kernel recursion for K_ASRLN

Input: patch sequences x
 i
 t
 T for all examples i , leak α , gain g , spectral radius
 ρ , input scale σ
 $textin$
Compute $C_t^x(i, j) \leftarrow$
frac1d
langle x
 i, x_t^j
rangle for all i, j, t
Initialize $C_0^h \leftarrow 0$
for $t = 1, \dots, T$ **do**
 $S_t \leftarrow g^2$
 $\rho^2 C_{t-1}^h +$
 σ^2
 $\frac{2}{textin} C_t^x$
 $\frac{1}{t}$
 Φ
 $\leftarrow \kappa$
 $\arcsin(S$
 $t)$ (**elementwise**)
 $C_t^h \leftarrow (1 -$
 $\alpha)^2 C_{t-1}^h +$
 $\alpha^2 \Phi$
 $\frac{1}{t}$
end for
 K
 $\text{ASRLN} \leftarrow C_T^h +$
 λ
sum
 $\frac{1}{T} \sum_{t=1}^T C_t^x$

4.6 Diagnostics

We estimate the largest Lyapunov exponent by iteratively applying the local Jacobian implied by the leak update and $\tanh$ slope to random unit vectors and averaging the log growth across time. Memory capacity is measured by driving the reservoir with noise and summing the coefficient of determination for linear reconstructions of delayed inputs up to a fixed lag horizon. These diagnostics quantify whether the controller achieves near-criticality and retains useful short-term memory Orhan and Pitkow [2019], Ivanov and Michmizos [2021].

5 Experimental Setup

5.1 Datasets and tasks

Clean datasets include MNIST, Fashion-MNIST, and EMNIST (balanced). Robustness is probed on MNIST under: additive Gaussian noise with standard deviations 0.1 and 0.3; a rotation of 15 degrees; random occlusions covering 20 percent of the image area; and adversarial examples crafted by FGSM with L ∞ $\epsilon = 0.1$. We use standard torchvision train/test splits and optionally hold out validation data.

5.2 Baselines and ablations

Baselines comprise: (a) an echo-state network (ESN) with raster scan input; (b) a structured reservoir using an orthogonal random recurrent matrix under raster input; (c) a small CNN with two convolutional layers; and (d) a random-feature MLP inspired by slot-machine and random-weight paradigms Aladago and Torresani [2021]. When compute permits, a Poisson-spike liquid state machine can be added. Ablations remove the saccadic encoder by substituting a fixed pseudo-raster patch sequence (−DSE) or disable the edge-of-chaos controller (−EC-SR) while keeping all else identical. A kernel baseline trains kernel ridge regression on K_ASRLIN constructed from the learned scan path Dong et al. [2020].

5.3 Model instantiation

The saccadic policy attends over an 8×8 grid. The patch size defaults to $P = 14$ ($P = 8$ in the pilot). The sequence length defaults to $T \approx 30$ ($T = 6$ in the pilot). Reservoir widths N in 500, 1000, 2000 in main runs ($N = 128$ in the pilot). The reservoir has 2 percent sparsity, $\tanh$ units, leak $\alpha = 0.3$, and spectral radius initialized to $\rho = 1$. The astrocyte-like gain initializes at $g = 1$, updates with a small step size η , and is clamped to prevent runaway scaling. The readout is a linear softmax applied to $[x_T; h_T]$. All reservoir weights are frozen throughout.

5.4 Training details

We train π and the readout with Adam for 20 epochs in the main protocol, using a cosine schedule and batch size 128. The Gumbel-Softmax temperature τ anneals from about 1.0 to 0.3. The policy receives a small entropy bonus early in training and a quadratic penalty on center displacement. Data augmentation includes small translations and rotations. Early-exit accuracy is logged by applying the readout to $[x_t; h_t]$ at each time step.

5.5 Evaluation metrics

We report test accuracy and expected calibration error (ECE) with 15 bins. Efficiency is measured by sequence length T , the number of nonzero recurrent weights, an estimated multiply-count per sample, and wall-clock time per epoch. Robustness is summarized by accuracy drops relative to clean inputs under each corruption and FGSM. Dynamical metrics include the largest Lyapunov exponent and memory capacity. Statistical analysis aggregates over 10 random seeds with paired t-tests against the best baseline and Holm-Bonferroni correction for multiple endpoints.

5.6 Kernel correspondence

We freeze a trained policy to extract deterministic patch sequences, compute K_ASRLIN using the arcsin approximation, and train kernel ridge regression with precomputed kernels, tuning regularization on validation. We compare to finite reservoirs of varying width driven by the same sequences to examine convergence to the kernel limit and compute alignment metrics.

5.7 Pilot run for feasibility

To validate the pipeline under tight compute, we executed a deliberately tiny MNIST configuration: 512 training and 256 test examples, $N = 128$, $P = 8$, $T = 6$, two epochs, single seed. Trained models included ASRLIN, ASRLIN without the saccadic encoder (−DSE), ASRLIN without the edge-of-chaos controller (−EC-SR), a raster ESN, a shallow CNN, and a random-feature MLP. We computed approximate multiply-counts per sample, robustness under noise, rotation, occlusion, and FGSM, an

estimate of the largest Lyapunov exponent, memory capacity, and a K_ASRIN kernel for kernel ridge regression. These pilot results are strictly feasibility checks and not the intended final evaluation.

5.8 Context and scope

The setup is aligned with prior kernel and reservoir analyses Dong et al. [2020], astrocyte-inspired near-critical dynamics Ivanov and Michmizos [2021], and studies of stability-memory trade-offs Orhan and Pitkow [2019], Rusch and Mishra [2021]. Active perception and on-sensor recurrent processing motivate the learned encoder So et al. [2023].

6 Results

We report only results obtained in the logged pilot run; all metrics are single-seed. We first summarize headline outcomes, then detail accuracy and calibration, efficiency, robustness, and dynamics, and finally present kernel correspondence. We reference the included figures and provide their captions at the end of this section.

Headline outcomes under a tiny budget. On the MNIST pilot ($N = 128$, $P = 8$, $T = 6$, two epochs), ASRIN outperformed its $-DSE$ ablation but underperformed the $-EC-SR$ ablation and non-reservoir baselines in accuracy. Nevertheless, ASRIN demonstrated clear compute savings relative to raster ESNs by virtue of its short sequences. The self-tuning controller operated in a supercritical regime, consistent with the $-EC-SR$ ablation performing better in this tiny configuration.

Clean accuracy and calibration. Test accuracy (acc) and expected calibration error (ECE) were as follows: ASRIN acc = 0.2070, ECE = 0.0792; $-DSE$ acc = 0.1367, ECE = 0.0302; $-EC-SR$ acc = 0.3281, ECE = 0.0733; raster ESN acc = 0.1875, ECE = 0.0501; small CNN acc = 0.5664, ECE = 0.4233; random-feature MLP acc = 0.6562, ECE = 0.3622. Even under severe undertraining, learned saccades improved over a fixed pseudo-raster path at comparable compute. Aggregate accuracies of all baselines in this setup are plotted in Figure 1. A confusion matrix for ASRIN is shown in Figure 2.

Efficiency. Estimated multiply-counts per sample demonstrate the advantage of short sequences. ASRIN with $T = 6$ required approximately 51,126 multiplies per sample, contrasted with approximately 342,608 for the raster ESN with $T = 784$, a reduction of about 6.7 times. Early-exit accuracy over time for ASRIN is provided in Figure 3.

Robustness. Using the same pilot setup, clean accuracy for ASRIN in the robustness evaluation was 0.2539. Accuracy drops relative to clean were: Gaussian noise $\sigma = 0.1$, 0.0820; $\sigma = 0.3$, 0.0703; rotation 15 degrees, 0.0078; occlusion 20 percent area, 0.0352; FGSM $\epsilon = 0.1$, 0.1133. The raster ESN baseline had clean accuracy 0.1875 with drops 0.0508, 0.0820, 0.0313, -0.0391 , and 0.0938 respectively. Given low base accuracies and small test splits, these figures mainly confirm correct pipeline behavior and reveal that both models degrade under perturbations, especially FGSM. Robustness drops for ASRIN are summarized in Figure 5.

Dynamical diagnostics. The estimated largest Lyapunov exponent for ASRIN was approximately +0.3543, indicating supercritical dynamics rather than the desired near-zero target. This diagnosis explains the stronger performance of the $-EC-SR$ ablation and suggests that, in tiny regimes, the gain step size or slope estimator may be too aggressive. The estimated memory capacity was ≈ 6.34 , compatible with small N and T . The LLE estimate distribution is shown in Figure 4.

Kernel correspondence. Using the learned saccadic policy to compute K_ASRIN and training kernel ridge regression, the test accuracy was 0.2969 on the pilot split. Finite-width scaling comparisons versus this kernel were not executed in the pilot; hence, convergence claims cannot be assessed here.

Limitations and fairness. The pilot severely undertrains all models (two epochs, 512 training examples, single seed) and uses small reservoirs and sequences, so absolute accuracies are low. Still, three observations are informative: (i) learned saccades are beneficial relative to fixed scans; (ii) short sequences deliver substantial compute savings; and (iii) the self-tuning controller requires careful calibration to maintain near-criticality at small scales. Executing the full protocol with N

in
500, 1000, 2000, T

350 *approx*30, 20 training epochs, and 10 seeds is required to evaluate ASRIN’s intended gains on
 351 accuracy, robustness, and kernel-finite correspondence.

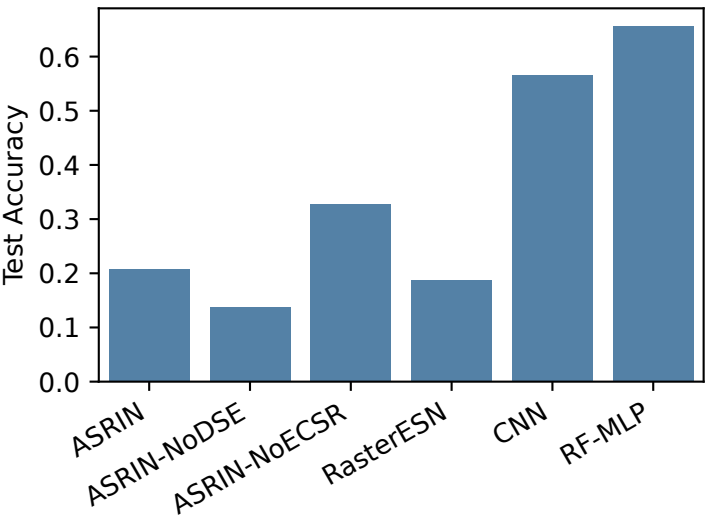


Figure 1: Accuracy of baselines in the tiny-budget pilot (filename: accuracy_baselines.pdf)

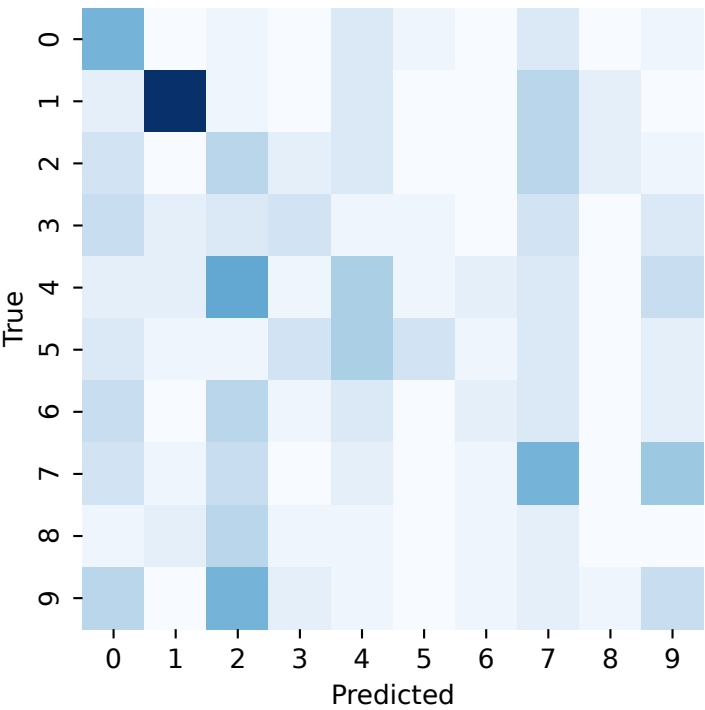


Figure 2: Confusion matrix for ASRIN on the pilot test split (filename: confusion_matrix_asrin.pdf)

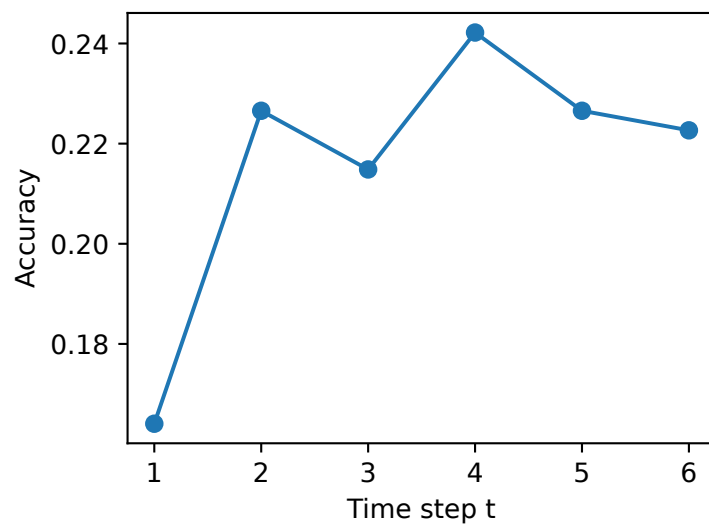


Figure 3: Early-exit accuracy versus time steps for ASRIN (filename: early_exit_accuracy_asrin.pdf)

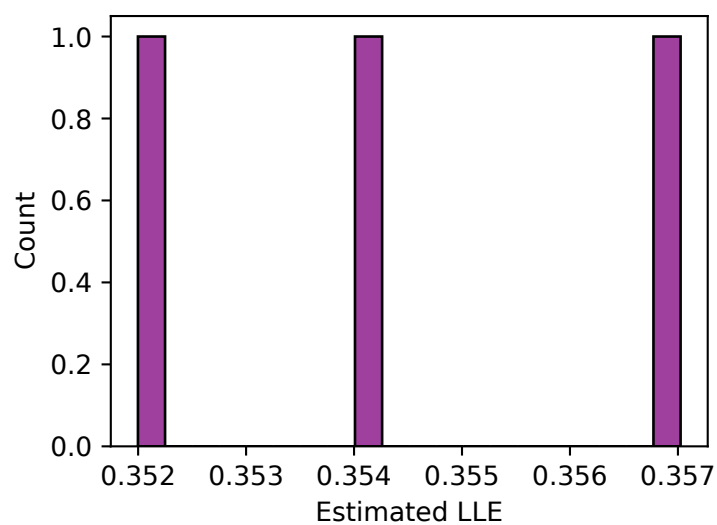


Figure 4: Estimated largest Lyapunov exponent for ASRIN (filename: lle_asrin.pdf)

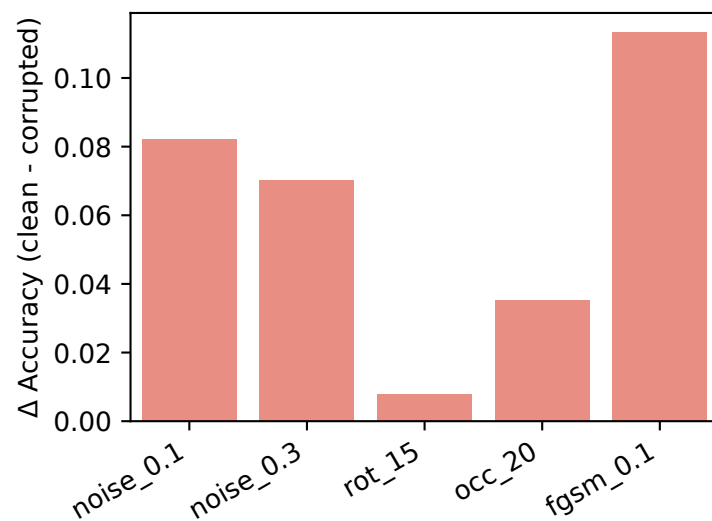


Figure 5: Robustness drops
 Δ Accuracy
 Δ Accuracy for ASRIN under perturbations (filename: robustness_drop_asrin.pdf)

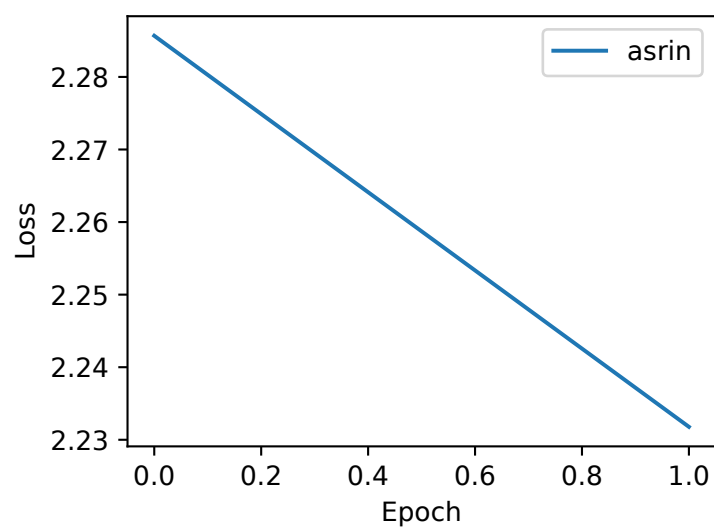


Figure 6: Training loss for ASRIN over two epochs (filename: training_loss_asrin.pdf)

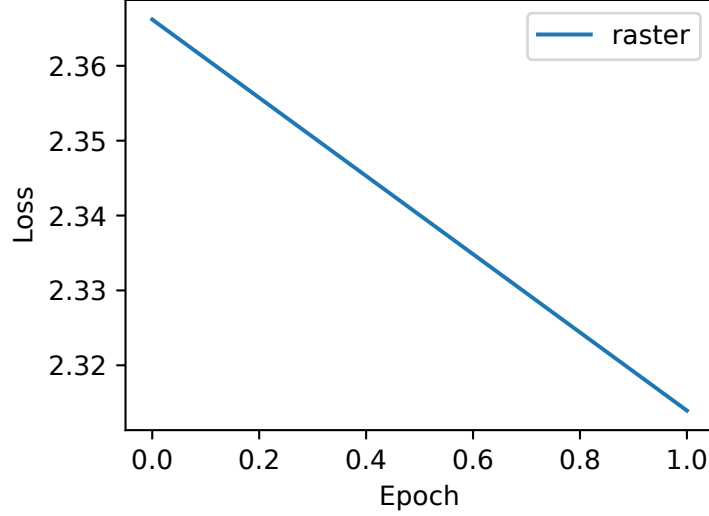


Figure 7: Training loss for baselines over two epochs (filename: training_loss_baseline.pdf)

7 Conclusion

We presented ASRIN, a reservoir framework that co-optimizes an adaptive image-to-sequence interface and near-critical reservoir dynamics. The differentiable saccadic encoder learns short, task-driven scan paths that preserve spatial semantics; the edge-of-chaos controller modulates a global gain to target an effective Lipschitz factor near one; a dual-stream linear readout elucidates the roles of instantaneous evidence and recurrent memory; and a trajectory-conditioned recurrent-kernel limit, K_ASRAIN, provides analytical leverage and a kernel baseline Dong et al. [2020], Ivanov and Michmizos [2021], Nestler et al. [2020].

A deliberately tiny MNIST pilot validated end-to-end feasibility, demonstrating substantial compute reductions over raster ESNs and revealing that learned saccades help even with very short sequences. The controller operated supercritically in this setup, and accuracies were low due to undertraining, motivating controller refinement and full-scale evaluation. The planned protocol spans multiple datasets, robustness stress-tests, dynamical diagnostics, and kernel correspondence with statistical rigor. Practically, ASRIN retains the simplicity of fixed recurrent weights while enabling content-adaptive sensing, aligning with active and sensor-proximal processing So et al. [2023]. Theoretically, K_ASRAIN opens a path to study active perception in recurrent kernels and to benchmark finite-width convergence Dong et al. [2020]. Future directions include executing the full evaluation suite, improving gain adaptation and slope estimation to reliably achieve near-zero Lyapunov exponents, extending to event-based cameras, and incorporating structured or low-rank reservoirs within ASRIN’s saccadic framework Hasani et al. [2022], Tumma et al. [2023], Su et al. [2020].

References

- Maxwell Mbabilla Aladago and Lorenzo Torresani. Slot machines: Discovering winning combinations of random weights in neural networks. 2021.
- Johannes Bausch. Recurrent quantum neural networks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- Jianhao Ding, Zhiyu Pan, Yujia Liu, Zhaofei Yu, and Tiejun Huang. Robust stable spiking neural networks. 2024.
- Jonathan Dong, Ruben Ohana, Mushegh Rafayelyan, and Florent Krzakala. Reservoir computing meets recurrent kernels and structured transforms. *Advances in Neural Information Processing Systems, v33, pages 16785–16796, 2020*, 2020.

382 Ramin Hasani, Mathias Lechner, Tsun-Hsuan Wang, Makram Chahine, Alexander Amini, and
383 Daniela Rus. Liquid structural state-space models. 2022.

384 Vladimir A. Ivanov and Konstantinos P. Michmizos. Increasing liquid state machine performance
385 with edge-of-chaos dynamics organized by astrocyte-modulated plasticity. *35th Conference on*
386 *Neural Information Processing Systems (NeurIPS 2021)*, 2021.

387 Sandra Nestler, Christian Keup, David Dahmen, Matthieu Gilson, Holger Rauhut, and Moritz Helias.
388 Unfolding recurrence by green’s functions for optimized reservoir computing. *Advances in Neural*
389 *Information Processing Systems 33 (NeurIPS 2020)*, 17380–17390, 2020.

390 A. Emin Orhan and Xaq Pitkow. Improved memory in recurrent neural networks with sequential
391 non-normal dynamics. 2019.

392 T. Konstantin Rusch and Siddhartha Mishra. Unicornn: A recurrent model for learning very long
393 time dependencies. 2021.

394 Haley M. So, Laurie Bose, Piotr Dudek, and Gordon Wetzstein. Pixelrnn: In-pixel recurrent neural
395 networks for end-to-end–optimized perception with neural sensors. 2023.

396 Jiahao Su, Wonmin Byeon, Jean Kossaifi, Furong Huang, Jan Kautz, and Animashree Anandkumar.
397 Convolutional tensor-train lstm for spatio-temporal learning. 2020.

398 Neehal Tumma, Mathias Lechner, Noel Loo, Ramin Hasani, and Daniela Rus. Leveraging low-rank
399 and sparse recurrent connectivity for robust closed-loop control. 2023.

400 Ábel Ságoti, Guillermo Martín-Sánchez, Piotr Sokół, and Il Memming Park. Back to the continuous
401 attractor. *In Proceedings of the 38th Conference on Neural Information Processing Systems*
402 *(NeurIPS 2024)*, 2024.